

By: Shoumik, Moneba, Judy, Ekam & Abrar

Group Project: Team AB
FreshRoute Schedule.pdf

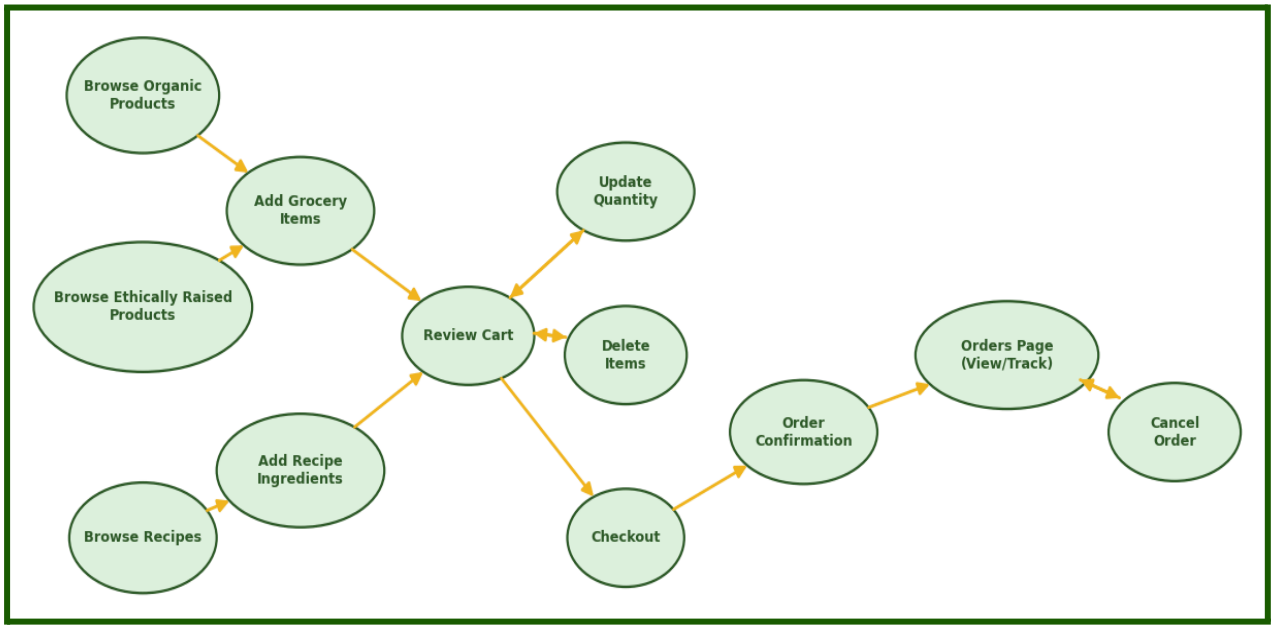
Team Members:

Name	Email	Student ID
Moneba Hanifi	moneba@my.yorku.ca	219721810
Shoumik Ahmed	shoumik@my.yorku.ca	219881598
Judy El Ghargomi	jelg21@my.yorku.ca	220227237
Ekambir Momi	ekam727@my.yorku.ca	219424407
Abrar Jawad Tarafder	abrart03@my.yorku.ca	219453497

3311 Section M Term W
Class Instructor: Ilir Dema
Sprint 3 Duration: March 23 - April 6, 2026

Network Diagram

The visual representation of task dependencies is attached below.



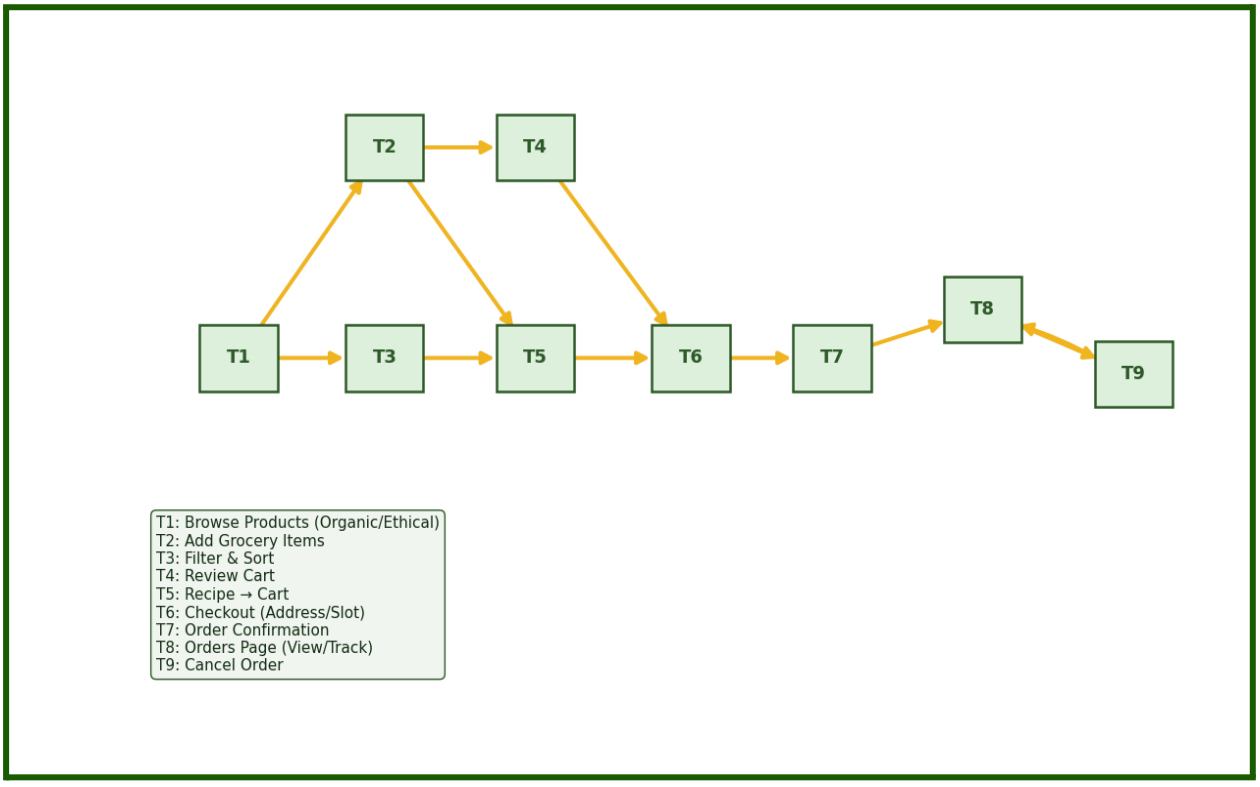


Diagram Description

The **top diagram** shows the user workflow of the FreshRoute system. Users can browse organic or ethically raised products and add grocery items to their cart. Users can also browse recipes and add recipe ingredients directly to the cart. These actions lead to the Review Cart stage, where users can update item quantities, delete items, or continue to Checkout. After checkout, the system shows an Order Confirmation, then allows the user to go to the Orders Page to view or track the order, and finally cancel the order if needed. This diagram represents the full customer flow by combining the earlier shopping/cart features with the new Sprint 3 order-management features.

The **bottom diagram** represents the development task dependency network. It shows the order in which major tasks depend on one another during implementation. The workflow begins with T1: Browse Products (Organic/Ethical). From there, the team builds T2: Add Grocery Items and T3: Filter & Sort. These features support later cart-related tasks such as T4: Review Cart and T5: Recipe → Cart. Once the cart flow is working, development moves to T6: Checkout (Address/Slot), followed by T7: Order Confirmation, T8: Orders Page (View/Track), and finally T9: Cancel Order. This diagram focuses on development order and dependency between tasks rather than on the user's step-by-step actions.

Critical Path

The critical path determines the main dependency chain that has the greatest impact on whether the sprint can be completed on time. For this sprint, the most important path was the sequence from the cart flow to checkout and then to post-order management. In our diagram, the main dependency path was:

T1 → T2 → T4/T5 → T6 → T7 → T8 → T9

In words, this means:

1. Browse Products
2. Add Grocery Items
3. Review Cart / Recipe to Cart
4. Checkout
5. Order Confirmation
6. Orders Page (View/Track)
7. Cancel Order

This was treated as the critical path because the Sprint 3 features depend on the order being completed successfully before it can be confirmed, viewed, tracked, or cancelled. Even though **Filter & Sort (T3)** is important, the order-management features cannot function unless the cart and checkout flow are already working correctly. Also, T6: Checkout depends on earlier cart-related work being stable, so any delay in cart review or recipe-to-cart integration would delay all Sprint 3 features later in the Sprint.

Strategies to Stay on Schedule

Core Flow First: We prioritized the shared shopping and cart flow before moving into the newer order-management features. We focused first on browsing, adding items, reviewing the cart, and checkout, because later features such as confirmation, tracking, and cancellation, all depend on those earlier steps working properly. This helped make sure that the Sprint 3 features were built on a stable foundation.

Parallel Work Across Features: We worked on multiple parts of the sprint in parallel whenever dependencies allowed. While some work focused on the cart and checkout flow, other work could continue on supporting features such as recipe-to-cart behaviour, order pages, and related improvements. This helped the sprint keep moving without making every task wait for one earlier task to fully finish.

Separate Development Branches: We used separate branches for major features so that different parts of the sprint could be developed and tested independently before merging. This reduced conflicts, made debugging easier, and helped us keep the implementation of checkout, order confirmation, order viewing/tracking, and cancel order more organized.

Regular Communication: We used standups and check-ins to catch blockers early, especially for tasks involving shared logic such as cart state, order data, and status transitions. This helped the team coordinate dependencies more effectively and respond faster when one task affected another.

Step-by-Step Integration: We tested and connected the system in the same order shown in the workflow diagram. We moved from browsing and cart actions into checkout, then into order confirmation, and finally into viewing, tracking, and cancelling orders. This reduced integration risk and helped us verify the most important user path first.

What Went Wrong

One challenge in this sprint was that several of the later features depended heavily on earlier shared functionality. Checkout, order confirmation, the orders page, and cancel-order features all relied on cart data and order data being passed correctly from one stage to the next. Because of this, even a small problem in the cart or checkout flow could affect multiple later features. Another issue was that the orders page and cancel-order functionality were closely connected. Since cancelling an order changes what should be shown on the order page, these features required extra coordination and testing to make sure the updated status was reflected correctly. The dependency diagram also shows that multiple paths eventually lead to checkout, making it one of the most sensitive and high-risk stages of the sprint. As a result, integration around checkout required more coordination than more isolated tasks.

Lessons Learned

Stabilize Shared Data Flow Early: We learned that shared data and transition logic should be stabilized as early as possible. Since many later features depend on the same cart and order information, early stability reduces the chance of repeated fixes later in the sprint.

Test Connected Features as One Flow: We learned that post-checkout features such as confirmation, tracking, and cancellation should not be treated as fully separate tasks during testing. They work as a single, connected flow, so testing them together provides a more accurate view of whether the system is functioning properly.

Prioritize Integration Earlier: We learned that features with strong dependencies should be integrated and validated earlier, not only near the end of the sprint. This helps reduce time pressure and gives more time to fix issues that affect multiple parts of the system.